4-bit Array Multiplier: RTL to GDSII with Open-Source Tools

Digital VLSI Assignment

Rashi Pandey
2026702015

Gauri Krishnan
2024102073

1 Introduction

This assignment involves designing and verifying a 4-bit unsigned array multiplier in Verilog, followed by taking the design through the RTL-to-GDSII flow using the SKY130 standard-cell library. The multiplier takes two 4-bit inputs and produces an 8-bit output, with 16 flip-flops used to register the inputs and output.

The design was implemented using open-source tools in the IIC-OSIC-TOOLS Docker environment. The report covers RTL design and verification, logic synthesis using Yosys, static timing analysis using OpenSTA, and physical design using LibreLane.

All source files, reports, and the commit history are available in the GitHub repository.

2 Design and Verification (Part A)

2.1 Architecture

Multiplying A and B is shift-and-add. For every bit b_i of B there is a partial product $pp_i[j] = a[j] \,\&\, b[i]$, shifted left by i. The hardware adds these rows one at a time (Fig. 1):

- Row 0 has only AND gates: $p[0] = pp_0[0]$ and $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$.
- Rows $i = 1, 2, 3$ are chains of four full adders. Adder j takes $pp_i[j]$, $r_{i-1}[j]$ and carry $c_i[j]$ (with $c_i[0] = 0$) and produces sum $s_i[j]$ and carry $c_i[j+1]$. Then $p[i] = s_i[0]$ and $r_i = \{c_i[4], s_i[3], s_i[2], s_i[1]\}$.
- The upper bits are $p[7:4] = r_3$.

In total there are 16 AND gates and 12 full adders.

`rtl/full_adder.v` implements $sum = a \oplus b \oplus c_{in}$ and $c_{out} = ab + a\,c_{in} + b\,c_{in}$ with `assign` statements. `rtl/mult_array.v` uses registers `a` and `b` (asynchronous active-low reset) for the 2 input numbers, builds the three adder rows with `generate` loops, and registers the 8-bit result in `p`. The `*` operator is not used in the design.

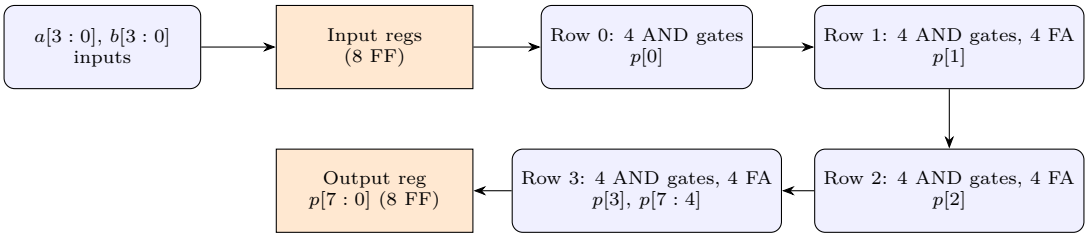


Figure 1: 16 AND gates, 12 full adders (three rows of four, ripple-carry inside each row) and 16 flip-flops (8 input, 8 output). Each row passes r_{i-1} to the next.

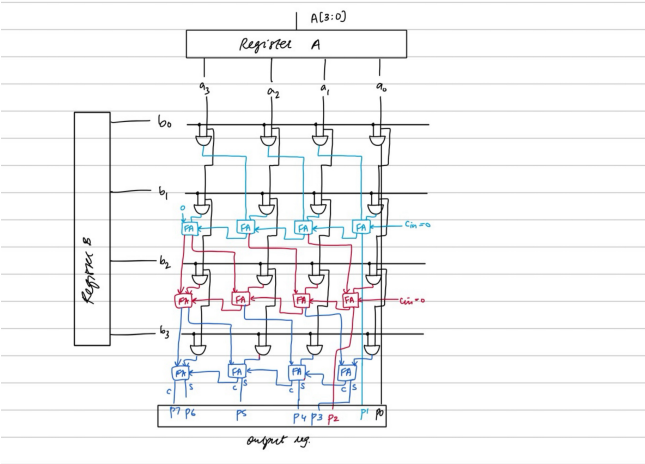


Figure 2: Block Diagram

2.2 Testbench and results

`tb/tb_mult.v` generates a 10 ns clock and holds `rst_n` low for the first two cycles. It applies all 256 combinations of $a = 0 \ldots 15$ and $b = 0 \ldots 15$. Inputs change on the falling clock edge to avoid a race with the active edge. After two rising edges (input register, then output register) it waits 4 ns and compares `p` with `a*b` (the `*` operator is used only in the testbench). It prints a line for every mismatch, one summary line at the end that tells you if all the testcases passed or not, and writes `mult.vcd`.

To check that the testbench really catches errors, `sum` in `full_adder.v` was temporarily changed to `a^b`. The testbench then reported 95 errors. The change was undone afterwards and the test passed again.

```
gauri@Gauri:~/eda/designs/array_multiplier$ vvp out
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:46: $finish called at 5149000 (1ps)
```

(a) Correct design: `PASS: 256 vectors checked, 0 errors`.

```
MISMATCH: a=15 b=14 p=154 expected=210
MISMATCH: a=15 b=15 p=181 expected=225
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:46: $finish called at 5149000 (1ps)
```

(b) Faulty full adder: `FAIL: 256 vectors checked, 95 errors`.

Figure 3: RTL simulation with the self-checking testbench.

2.3 Waveform and latency

The example used takes the last digit of Rashi's roll number (5): $A = 5 + 5 = 10$ and $B = 7$, so $A \times B = 70$.

Figure 4 shows the multiplication of $A = 10$ and $B = 7$ in the GTKWave simulation waveform, with values displayed in decimal.

- **Marker A (3370 ns):** The testbench changes b from 6 to 7 at the falling clock edge. The output still shows the previous result, $10 \times 6 = 60$.
- **Marker B (3375 ns):** At the first rising clock edge, the inputs are captured into the input registers. The output remains $p = 60$.
- **Marker C (3385 ns):** At the second rising clock edge, the output register captures the new product, $p = 70$, confirming that $10 \times 7 = 70$.

The result is captured at the second rising clock edge after the inputs are applied, which confirms the 2-cycle latency.

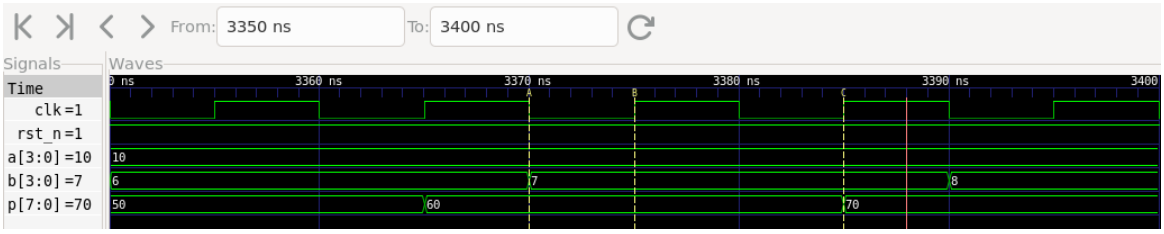


Figure 4: Waveform for $a = 10$, $b = 7$

2.4 Hand-worked example

$A = 1010_2$ (10) and $B = 0111_2$ (7)

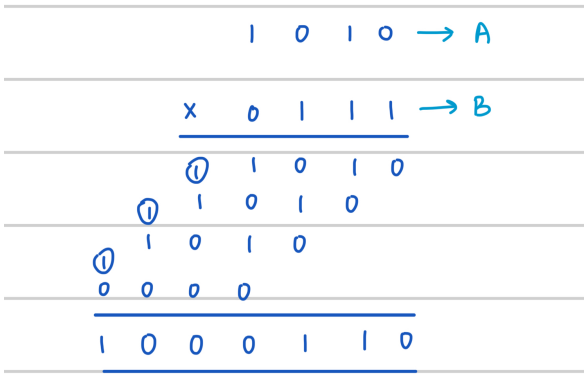


Figure 5: Hand-worked example $10 \times 7 = 70$.

3 Synthesis (Part B)

3.1 Logic Synthesis using Yosys

Yosys is an open-source synthesis tool that converts RTL Verilog into a gate-level netlist. It optimizes the design and maps the logic to standard cells from the SKY130 technology library.

The synthesis process was carried out using the `synth/synth.tcl` script, which performs the following steps:

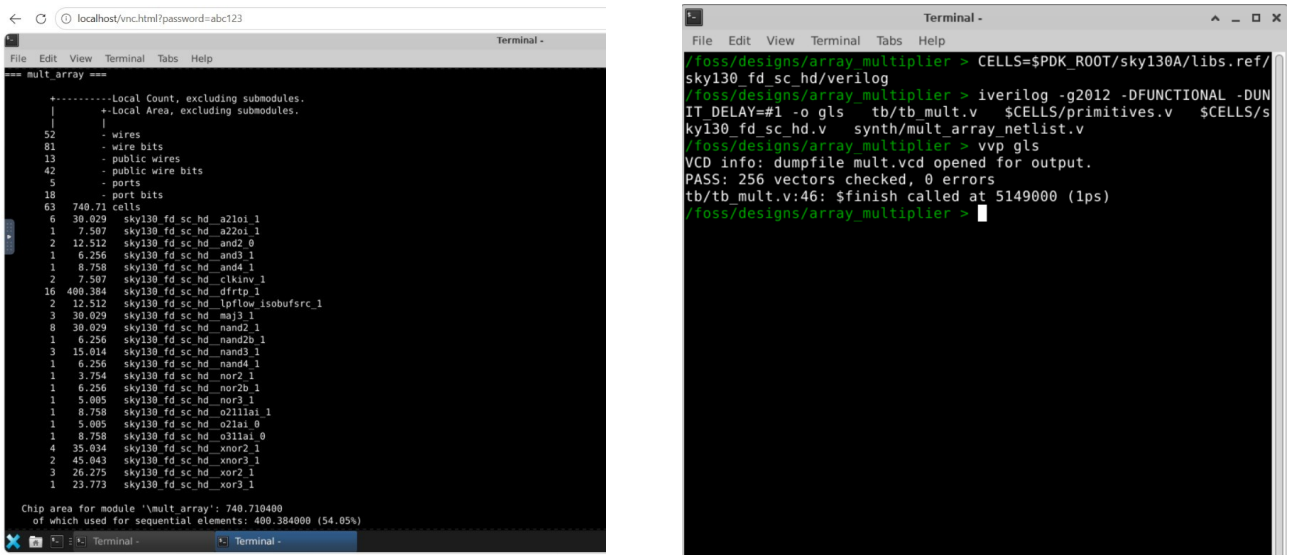
1. `read_verilog`: Reads the Verilog files containing the multiplier and full adder modules.
2. `synth -top mult_array -flatten`: Performs generic synthesis and flattens the design into a single module.
3. `dfflibmap`: Maps the flip-flops to appropriate SKY130 standard cells.
4. `abc`: Optimizes and maps the combinational logic to SKY130 gates.
5. `opt_clean -purge`: Removes unused wires and unnecessary logic.
6. `stat -liberty`: Generates a report showing the number of cells used and their total area.
7. `write_verilog`: Saves the synthesized design as a gate-level Verilog netlist.

Table 1: Synthesis results (Yosys, SKY130 `hd`).

Metric	Value
Total number of cells	63
Number of flip-flops (`dfrtp_1`)	16
Total cell area	740.71 μm^2
Flip-flop area (16 $\times$ 25.024 μm^2)	400.38 μm^2
Flip-flop area (% of total)	54.05 %
3 most-used cell types	`dfrtp_1` (16), `nand2_1` (8), `a21oi_1` (6)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Note: `dfrtp` denotes a positive-edge-triggered D flip-flop with active-low reset in the SKY130 library (cool!)

The synthesized netlist was simulated using the same testbench as in Part A, along with the SKY130 standard-cell models. The flag `-DFUNCTIONAL` enables the functional behavior of the SKY130 standard cells, and `-DUNIT_DELAY=#1` enables a simplified 1-unit delay in the relevant cell models.
All 256 input combinations passed, confirming that the synthesized design produces the same results as the original RTL implementation.



(a) `mult_array_stat.rpt`: 63 cells, 740.71 μm^2.

(b) Gate-level simulation: All 256 vectors pass.

Figure 6: Synthesis report and netlist simulation.

4 Static Timing Analysis (Part C)

4.1 Static Timing Analysis using OpenSTA

Static Timing Analysis (STA) uses OpenSTA to check whether the synthesized multiplier meets the required clock timing. It uses the gate-level netlist generated by Yosys in Part B (`synth/mult_array_netlist.v`) and the SKY130 Liberty library, which contains the cell delays and setup time requirements. The interesting thing about STA is that it analyses timing without simulating individual input combinations.

The `sta/sta.tcl` script loads the netlist and timing library, defines a clock period of 10 ns, and applies input and output delays of 1 ns. OpenSTA then calculates the arrival time and required time for each constrained timing path. The difference between these is called *slack*:

$$\text{Slack} = \text{Required Time} - \text{Arrival Time} \tag{1}$$

A positive slack indicates that the signal arrives before its timing deadline, while a negative slack indicates a timing violation. The *worst slack* is the smallest slack among all analyzed paths and identifies the most timing-critical path.

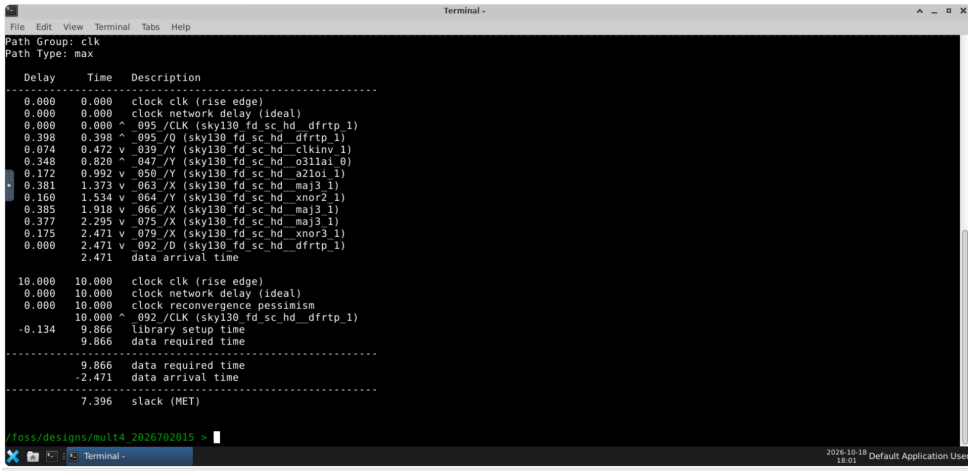


Figure 7: Critical path report from `a_r[1]` to `p[6]`: data arrival 2.471 ns, slack 7.396 ns.

As shown in Figure 7, OpenSTA reported a data arrival time of 2.471 ns for the critical path. With a clock period of 10 ns and a flip-flop setup time of 0.134 ns, the required arrival time is 9.866 ns. Therefore,

$$\text{Worst Slack} = 9.866 - 2.471 \approx 7.396 \text{ ns} \tag{2}$$

The minimum clock period and maximum operating frequency were calculated as:

$$T_{\text{min}} = T - \text{Worst Slack} = 10 - 7.396 = 2.604 \text{ ns} \tag{3}$$

$$F_{\text{max}} = \frac{1000}{T_{\text{min}}} = \frac{1000}{2.604} \approx 384.0 \text{ MHz} \tag{4}$$

STA was then repeated with a clock period of 2.61 ns (2.604 ns rounded up to 0.01 ns), which gave a slack of +0.01 ns, close to zero, so the design is at its limit.

Table 2: Timing results (typical corner).

Metric	Value
Worst slack at $T = 10$ ns	7.396 ns
T_{min}	2.604 ns
F_{max}	384.0 MHz
Slack at $T = T_{min}$ ($T = 2.60$ ns)	+0.01 ns
Critical path start point	`a_r[1]` (flip-flop `_095_`)
Critical path end point	`p[6]` (flip-flop `_092_`)
Cells on the critical path	8 combinational (10 with the two flip-flops)
Latency $= 2 \times T_{min}$	5.208 ns
Throughput	384.0 million results/s

Table 3: Critical path (register to register) at $T = 10$ ns.

Instance	Cell	Delay (ns)	Arrival (ns)
`_095_` (`a_r[1]`), clk→Q	`dfrtp_1`	0.398	0.398
`_039_`	`clkinv_1`	0.074	0.472
`_047_`	`o311ai_0`	0.348	0.820
`_050_`	`a21oi_1`	0.172	0.992
`_063_`	`maj3_1`	0.381	1.373
`_064_`	`xnor2_1`	0.160	1.534
`_066_`	`maj3_1`	0.385	1.918
`_075_`	`maj3_1`	0.377	2.295
`_079_`	`xnor3_1`	0.175	2.471
`_092_` (`p[6]`), D input	`dfrtp_1`	–	2.471

Library setup time 0.134 ns, data required time 9.866 ns, slack 7.396 ns.

The critical path runs from input register `a_r[1]` to output register `p[6]`, with a worst setup slack of 7.396 ns. It passes through 8 combinational cells: `clkinv_1`, `o311ai_0`, `a21oi_1`, `maj3_1`, `xnor2_1`, `maj3_1`, `maj3_1`, and `xnor3_1`, in that order.

Of these, 5 cells are mainly associated with full-adder logic: three `maj3` cells for carry generation and two XNOR cells for sum logic. The remaining cells implement inversion and combined AND/adder logic introduced during technology mapping.

For a 4-bit multiplier, the theoretical critical-path estimate is approximately $2N = 8$ full-adder delays. Although the actual path also contains 8 combinational cells, these are not equivalent to 8 complete full adders. The difference arises because Yosys simplifies and combines the original logic during synthesis.



(a) $T = 10$ ns: worst slack 7.40 ns. **(b)** $T = 2.61$ ns: worst slack 0.01 ns.

Figure 8: OpenSTA worst slack at $T = 10$ ns and at $T \approx T_{min}$.

5 Physical Design (Part D)

5.1 Setup and configuration

The multiplier was taken from RTL to GDSII with LibreLane on SKY130A (`sky130_fd_sc_hd`) using `pnr/config.json`:

```
{
  "DESIGN_NAME": "mult_array",
  "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 6.5.,
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 150, 150]
}
```

The die size was set to 150×150 μm, as specified in the assignment for this small design.

The initial clock period was calculated using the assignment's formula:

$$\text{CLOCK_PERIOD} = 1.2 \times T_{\text{min}} = 1.2 \times 2.604 = 3.125 \text{ ns}$$

Rounding up to the next 0.5 ns gives 3.5 ns. However, this period failed the setup timing check at the slow corner (`ss_100C_1v60`). Unlike Part C, which only considered typical conditions, LibreLane also checks timing at slower operating conditions, where the cells take longer to propagate signals.

The clock period was therefore increased and the flow was repeated until setup timing passed, in steps of 0.5 ns up to 5.0 ns and then in two steps of 1.0 ns (5.0 to 6.0 to 7.0 ns). 7.0 ns passed first; 6.5 ns and 5.5 ns were then also tried to find a shorter passing period. 6.5 ns passed and 5.5 ns failed (-0.142 ns), so 6.5 ns is the final value.

Table 4: `CLOCK_PERIOD` attempts (all with `sky130A`). All failures were in the `ss_100C_1v60` corner.

Run	`CLOCK_PERIOD` (ns)	Worst setup slack (ns)	Result
1	3.5	-2.129	failed
2	4.0	-1.629	failed
3	4.5	-1.139	failed
4	5.0	-0.642	failed
5	5.5	-0.142	failed
6	6.0	-0.064	failed
7	6.5	$+0.085$	passed, final

Up to 5.0 ns each 0.5 ns of additional period improved the worst slack by almost exactly 0.5 ns, because the critical path delay stays the same.

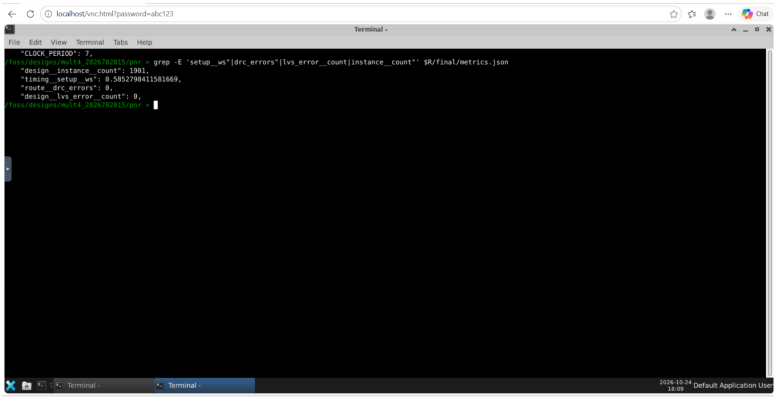


Figure 9: Final run (6.5 ns): setup slack +0.085 ns, DRC 0, LVS 0, 1901 instances.

5.2 Final results

Table 5: Physical design results of the final run (`CLOCK_PERIOD` = 6.5 ns).

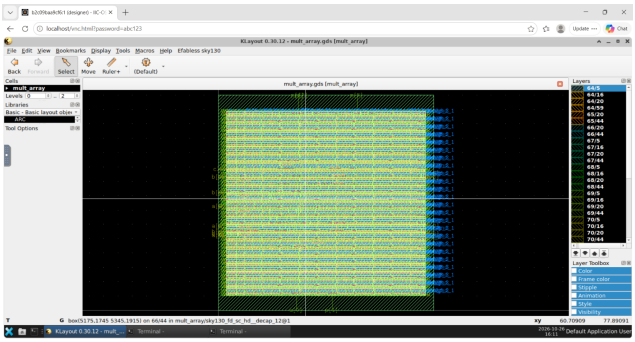
Metric	Value
`CLOCK_PERIOD` used	6.5 ns
Number of runs needed	7
Core area	17759.5 μm^2
Cells after synthesis (Part B)	63
Cells after place and route	1901
Kinds of cells added	tap (245), fill/decap (1566), clock buffers (3), timing-repair buffers (19)
Wirelength	2405 μm
Setup slack (worst corner, `ss_100C_1v60`)	+0.085 ns
Post-route $F_{max} = 1000/(6.5 - 0.085)$	155.9 MHz
F_{max} from Part C	384.0 MHz
DRC errors	0
LVS errors	0

Max Freq: The post-route $F_{\max}$ is 155.9 MHz whereas we obatined 384.0 MHz in Part C.

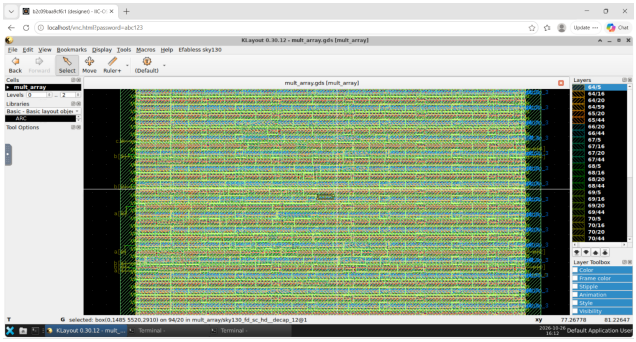
Cell breakdown: The 1901 instances are 335 standard cells plus 1566 fill cells. The 335 standard cells are 245 tap cells and 90 functional cells (16 flip-flops, 3 inverters, 49 multi-input combinational cells, 19 timing-repair buffers and 3 clock buffers).

Verification:

- DRC: During routing, the number of DRC violations changed across iterations (9, 3, 4, and finally 0) as LibreLane adjusted the wire connections. The final layout passed the routing, Magic, and KLayout DRC checks with zero violations.
- LVS: the device, net, property, unmatched-device, unmatched-net and unmatched-pin counts are all 0.
- Timing: the setup slack is positive in all corners. The worst is +0.085 ns in `max_ss_100C_1v60`; in the typical corner it is +3.29 ns.



(a) Whole 150 × 150 μm die.



(b) Zoom: cell rows, metal wires and a `decap_12` cell.

Figure 10: Final layout of `mult_array` in KLayout.

6 Answers to Questions

Q1

Synthesis produced 63 standard cells, of which 16 are flip-flops, leaving 47 combinational logic cells. This differs from the original 16 AND gates and 12 full adders because most SKY130 standard cells have a single output, whereas a full adder produces both a sum and carry. Therefore, a full adder generally requires multiple cells, such as XOR/XNOR gates for the sum and MAJ3 (majority) gates for the carry. Yosys also simplifies the logic by combining operations into complex gates and reducing the first full adder in each row to half-adder logic, since its carry-in is always zero.

Q2

The critical path starts at input register `a_r[1]` and ends at output register `p[6]`. It passes through 8 combinational cells, with a total data arrival time of 2.471 ns, including 0.398 ns of clock-to-Q delay. The theoretical estimate is approximately $2N = 8$ full-adder delays for a 4-bit multiplier. Although the numbers match, only 5 of the 8 cells directly implement adder operations, since Yosys combines and optimizes the logic. The actual combinational delay is $2.471 - 0.398 = 2.073$ ns. Dividing this by the theoretical estimate of 8 full-adder delays gives an average of approximately 0.26 ns per full-adder stage.

Q3

Flip-flops take $400.38/740.71 = 54.05\,\%$ of the cell area. Each `dfrtp_1` (with asynchronous reset) occupies approximately 25.02 μm^2, while the 47 logic cells average only about 7.2 μm^2. The multiplier is tiny, but all 16 inputs and outputs are registered, so the registers are a large share of the area.

Q4

The post-route maximum frequency is 155.9 MHz, compared to 384.0 MHz obtained in Part C. This reduction is mainly due to three factors. First, Part C uses typical operating conditions, while Part D considers the slow corner (`ss_100C_1v60`), where gates operate more slowly. Second, physical routing introduces wire resistance and capacitance, increasing signal delay. Third, the clock tree introduces additional delays and clock skew, whereas Part C assumes an ideal clock.

The physical design flow also adds several types of cells:

- **Tap cells (245):** Connect semiconductor wells to power or ground to prevent latch-up.
- **Fill/decap cells (1566):** Fill gaps between cells and help maintain stable power connections. Decap cells also reduce supply noise.
- **Clock buffers (3):** Distribute the clock signal to the flip-flops while minimizing clock skew.
- **Timing-repair buffers (19):** Improve signal transitions and timing after placement.

Q5

DRC (Design Rule Check) verifies that the physical layout follows manufacturing rules, such as minimum wire width and spacing. LVS (Layout Versus Schematic) checks whether the connections in the final layout match the intended circuit netlist. Both checks are necessary because a layout may follow manufacturing rules but contain incorrect connections, or have correct connections but violate manufacturing rules. Our final design passed both checks with zero errors.

Q6

For an 8-bit array multiplier, the number of AND gates increases from $4^2 = 16$ to $8^2 = 64$, while the number of full adders increases from $4(4-1) = 12$ to $8(8-1) = 56$.

The theoretical critical path grows from approximately $2 \times 4 = 8$ to $2 \times 8 = 16$ full-adder delays, so the combinational delay roughly doubles. Using the measured delay of 2.073 ns for the 4-bit design, the estimated combinational delay for the 8-bit multiplier is:

$$T_{\text{comb}} \approx 2 \times 2.073 = 4.146 \text{ ns}$$

Including the clock-to-Q delay of 0.398 ns and setup time of 0.134 ns:

$$T_{\text{min}} \approx 4.146 + 0.398 + 0.134 = 4.678 \text{ ns}$$

$$F_{\text{max}} \approx \frac{1000}{4.678} \approx 214 \text{ MHz}$$

This is only an approximate estimate, assuming similar gate delays and synthesis optimizations for the larger multiplier.

7 Conclusion

The 4-bit array multiplier passed all 256 test cases in both RTL and gate-level simulation. Yosys synthesis produced 63 cells with a total area of 740.71 μm^2, of which 54.05% was occupied by flip-flops.

OpenSTA estimated a maximum frequency of 384 MHz under typical conditions. After physical design using LibreLane, the final layout passed with zero DRC and LVS errors and a positive setup slack of 0.085 ns at a 6.5 ns clock period after eight runs. The post-route maximum frequency was 155.9 MHz, mainly due to slower operating conditions and physical routing delays.

8 Contributions

Gauri: Part A, Part B, Part C, Report
Rashi: Report, Part A, Part B, Part C, Part D, README